

Characterising On-Device AI Failure Modes Under Joint Fault Injection and Runtime Observability

Muhammad Assad Ullah

University of Engineering and Technology Taxila, Pakistan

ORCID: [0009-0007-2290-304X](https://orcid.org/0009-0007-2290-304X) | asadullahaj602@gmail.com

Code	github.com/assassinaj602/edgepulse
SATE AI	DOI 10.5281/zenodo.23250418
EdgePulse	DOI 10.5281/zenodo.23248718
Dataset	github.com/assassinaj602/edgepulse/tree/main/research/experiment/results
Date	October 2026

ABSTRACT

Deploying AI models on edge devices exposes machine learning runtimes to hardware pressure that cloud deployments never face: thermal throttling, volatile system memory, and corrupted input tensors. Prior work addresses fault injection and runtime observability separately. No empirical study has combined them in a single pipeline across heterogeneous model architectures.

We present a joint empirical study combining SATE AI (software-level fault injection) and EdgePulse (non-intrusive runtime observability) on a physical Android handset (Tecno CH7n, MediaTek Helio G35, Android 12). Across 280 real-device inference traces covering three model runtimes (TFLite, ONNX Runtime, and llama.cpp GGUF) and four execution scenarios (baseline, memory pressure, malformed input, and thermal stress), we find three results:

1. Thermal stress produces +486.8% latency regression in MobileNetV3 TFLite and +406.6% in ResNet-18 ONNX, while `PowerManager.currentThermalStatus` reports `nominal` in 100% of 70 thermal stress runs, replicating and extending the thermal-API blind spot.
2. Malformed input produces a bimodal failure signature in TinyLlama 1.1B GGUF: 8 of 10 runs complete in 807-2,680 ms, while 2 of 10 enter a 20-second pathological generation loop (19,702 ms and 19,963 ms), giving a standard deviation of 7,244 ms.
3. Vision models and LLMs have inverted fault-tolerance profiles. Vision models are thermal-sensitive but resilient to malformed input. LLMs are malformed-input-sensitive but less thermal-sensitive.

Both tools are open-source and available at pub.dev/packages/sate_ai and pub.dev/packages/edgepulse under the MIT license.

1. Introduction

Quantized LLMs and compact computer vision architectures are increasingly deployed inside mobile applications. Runtimes like TFLite [3], ONNX Runtime [4], and llama.cpp [5] make local inference viable on consumer hardware, cutting cloud costs and enabling offline use. Moving inference to mobile devices, however, shifts reliability responsibility onto application developers. Mobile SoCs operate under passive thermal budgets, strict RAM ceilings, and energy constraints that have no equivalent in server environments.

Three hardware-level failure modes are particularly relevant. First, sustained dense matrix operations raise SoC junction temperatures until the kernel enforces DVFS clock reductions or disables performance cores [9]. Second, KV-cache growth during LLM decoding can exceed available RAM, triggering process termination by Android's Low Memory Killer or iOS Jetsam. Third, corrupted or boundary-violating input tensors can cause numerical instability or pathological generation loops in autoregressive models.

Prior work addresses these concerns in isolation. SATE AI [1] implements software-level fault injection at the model runtime boundary. EdgePulse [2] captures per-inference telemetry (RSS, CPU, battery current, thermal state) on Android and iOS. Neither has been applied jointly in a single experiment. Without that combination, developers lack empirical data describing how specific fault types manifest differently across model families under physical execution constraints.

1.1 Contributions

1. **Joint reliability pipeline:** A closed-loop methodology integrating SATE AI fault injectors with EdgePulse telemetry tracing inside a Flutter integration test framework on physical hardware.
2. **280-trace empirical dataset:** Three model runtimes (MobileNetV3 TFLite, ResNet-18 ONNX, TinyLlama 1.1B GGUF) across four controlled scenarios, all collected on real Android hardware.
3. **Three novel findings:** Thermal-API blind spot replicated and extended; bimodal LLM failure under malformed input discovered; inverted fault-tolerance profiles characterised between vision models and LLMs.

2. Background and Related Work

2.1 On-Device Runtimes

TFLite [3] supports 8-bit integer quantization [6] and hardware delegate acceleration. ONNX Runtime [4] provides cross-platform graph optimization across mobile and

desktop. llama.cpp/GGUF [5] targets quantized LLMs (4-bit Q4_K_M format) on ARM CPU architectures. Each runtime presents distinct memory and compute profiles, making cross-runtime comparison non-trivial.

2.2 SATE AI: Fault Injection

Traditional chaos engineering tools like Chaos Mesh and Gremlin target infrastructure-level faults (network drops, process kills) that do not reach the model runtime boundary. SATE AI [1] fills this gap with 11 domain-specific fault injectors and 8 model adapters, operating entirely within the application process and requiring no root access or specialized hardware. The framework is validated by 246 automated tests and holds pub.dev's maximum quality score of 160/160.

2.3 EdgePulse: Runtime Observability

IDE profilers (Android Studio Profiler, Xcode Instruments) capture detailed hardware counters but require an interactive debug session and cannot run in CI. EdgePulse [2] captures per-inference `InferenceTrace` records (RSS via `Debug.getPss()`, thermal state via `PowerManager.currentThermalStatus`, battery current via `BatteryManager.CURRENT_NOW`, CPU from `/proc/stat`) with no USB connection or debug session required. A prior single-tool study [2] identified that `PowerManager.currentThermalStatus` reported `nominal` throughout a 15-minute TinyLlama session that produced a +16.5% latency regression.

2.4 The Research Gap

No prior study has applied fault injection and runtime telemetry jointly across multiple model families. This paper fills that gap, quantifying how specific injected faults produce physically measurable degradation signals across TFLite, ONNX, and GGUF runtimes on the same device.

3. Methodology

3.1 Joint Pipeline Architecture

```
SATE AI Fault Injector (MemoryPressureInjector /
MalformedInputInjector / Thermal loop) | v Model Runtime Under
Stress (TFLite / ONNX Runtime / llama.cpp GGUF) | v EdgePulse
PulseRunner (pulse.traceMany()) - PlatformMetricCollector: RSS,
CPU, Battery, Thermal - Per-inference InferenceTrace records | v
CSV Dataset + Statistical Analysis
```

For each execution: the SATE AI injector activates before inference (pre-allocating memory, generating corrupted tensors, or sustaining back-to-back load). EdgePulse's `pulse.traceMany()` wraps each inference call, capturing pre- and post-inference memory snapshots, thermal state, battery current, and wall-clock latency in a single `InferenceTrace` record.

3.2 Hardware

All experiments were performed on a physical Tecno CH7n handset: MediaTek Helio G35 SoC (8-core ARM Cortex-A53 at up to 2.3 GHz), 4 GB LPDDR4X RAM, Android 12 (API 31, HiOS). No mock metric collectors or simulated environments were used.

3.3 Models

1. **MobileNetV3-Small (TFLite):** Quantized image classification model (9.8 MB, approximately 4.5M parameters) ^[7].
2. **ResNet-18 (ONNX Runtime):** Image classification backbone (44.7 MB, approximately 11.7M parameters) ^[10].
3. **TinyLlama 1.1B Chat (GGUF Q4_K_M):** 4-bit quantized autoregressive language model (638 MB, 512-token context window) ^[8].

3.4 Scenarios

- **Baseline:** Clean execution under nominal system conditions.
- **Memory Pressure:** SATE AI `MemoryPressureInjector` pre-allocates 150 MB of native heap before each inference.
- **Malformed Input:** SATE AI `MalformedInputInjector` generates boundary-violating and corrupted input payloads.
- **Thermal Stress:** Back-to-back inference loops (5 iterations for vision models, 2 for GGUF) generate real SoC heating before the measured call.

3.5 Sample Sizes and Statistical Tests

30 measured runs per scenario for MobileNetV3 and ResNet-18 (120 traces each); 10 runs per scenario for TinyLlama (40 traces), giving 280 total traces. For each faulted scenario vs. baseline within each model, we compute Mann-Whitney U statistic, two-tailed p-value, and rank-biserial correlation (r) as the effect size. All tests are non-parametric because normality is not assumed for execution-time distributions.

4. Results

4.1 Baseline Characteristics

Model	Format	N	Mean Latency	Std Dev	Median	Peak RSS
MobileNetV3-Small	TFLite	30	132.9 ms	12.7 ms	128.0 ms	291.0 MB
ResNet-18	ONNX	30	141.4 ms	30.8 ms	127.0 ms	478.0 MB
TinyLlama 1.1B	GGUF	10	1,649.6 ms	136.1 ms	1,623.0 ms	1,018.2 MB

Table 1: Baseline inference characteristics on Tecno CH7n. Peak RSS spans 708.5 MB across model classes, confirming authentic native memory allocations.

4.2 Full Fault Scenario Results

Model	Scenario	N	Mean (ms)	Std Dev	Change	p-value	Effect r
MobileNetV3 TFLite	Baseline	30	132.9	12.7	--	--	--
	Memory Pressure	30	140.8	9.3	+5.9%	0.0001	0.654
	Malformed Input	30	159.7	15.6	+20.1%	0.0000	0.812
	Thermal Stress	30	779.8	42.9	+486.8%	0.0000	1.000
ResNet-18 ONNX	Baseline	30	141.4	30.8	--	--	--
	Memory Pressure	30	154.7	27.5	+9.4%	0.0115	0.382
	Malformed Input	30	145.0	21.2	+2.5%	0.0993 (n.s.)	0.246
	Thermal Stress	30	716.4	56.5	+406.6%	0.0000	1.000
TinyLlama GGUF	Baseline	10	1,649.6	136.1	--	--	--
	Memory Pressure	10	1,628.5	147.4	-1.3%	0.9097 (n.s.)	-0.060
	Malformed Input	10	5,379.7	7,244.2	+226.1%	0.2899	0.320
	Thermal Stress	10	3,309.5	127.1	+100.6%	0.0002	1.000

Table 2: Per-scenario performance metrics and Mann-Whitney U test results. Highlighted rows are the paper's three key findings. n.s. = not statistically significant at $p < 0.05$.

4.3 Finding 1: Thermal-API Blind Spot Replicated and Extended

Key result: MobileNetV3 latency increased +486.8% under thermal stress (132.9 ms to 779.8 ms, $p < 0.0001$, $r = 1.000$). ResNet-18 increased +406.6% (141.4 ms to 716.4 ms, $p < 0.0001$, $r = 1.000$). In 100% of 70 thermal stress runs, `PowerManager.currentThermalStatus` returned `THERMAL_STATUS_NONE` (nominal).

Both vision models degraded by roughly 5x under thermal stress. The effect size of $r = 1.000$ means there is zero overlap between the baseline and thermal-stress latency distributions: every single thermal-stress run was slower than every single baseline run. TinyLlama also degraded significantly (+100.6%, $p = 0.0002$), though the magnitude is smaller because the model's single-threaded generation loop is less susceptible to core-count reduction than the parallel convolution workloads of the vision models.

The OS thermal API reported nominal throughout. This extends the finding from the prior EdgePulse study [2], which observed the same blind spot during sustained single-model load. The current result shows it holds across actively injected thermal stress and across two distinct model families. On budget OEM Android distributions, `PowerManager.currentThermalStatus` maps to extreme battery temperature thresholds rather than real-time SoC junction registers, making it unreliable for application-layer performance monitoring.

4.4 Finding 2: Bimodal LLM Failure Under Malformed Input

Key result: TinyLlama GGUF under malformed input shows mean latency of 5,379.7 ms with std dev 7,244.2 ms. Raw distribution: [807, 1412, 1608, 1615, 1669, 1800, 2541, 2680, 19702, 19963] ms. Two distinct clusters with no overlap.

Eight of ten runs completed between 807 ms and 2,680 ms, close to the baseline median. Two runs took 19,702 ms and 19,963 ms, roughly 12x the median. The bimodal shape is not noise. The p-value is 0.2899 (not significant) because $N = 10$ is too small to achieve significance against a high-variance distribution, but the pattern itself is unmistakable.

The mechanism: when the malformed input injector provides boundary-violating or binary-garbage payloads, the llama.cpp tokenizer produces token sequences that cause the sampler to miss EOS (end-of-sequence) tokens. The generation loop then continues until reaching the maximum context boundary, producing 20 seconds of wall-clock blocking on a thread that the application cannot interrupt. Vision models show no

equivalent behaviour: MobileNetV3 increased 20.1% (statistically significant but bounded) and ResNet-18 showed no significant change at all (+2.5%, $p = 0.0993$).

4.5 Finding 3: Inverted Fault-Tolerance Profiles

The two findings above, taken together, reveal a structural difference in how vision models and LLMs fail:

Fault Type	MobileNetV3 TFLite	ResNet-18 ONNX	TinyLlama GGUF
Memory Pressure	+5.9% (sig.)	+9.4% (sig.)	-1.3% (n.s.)
Malformed Input	+20.1% (sig.)	+2.5% (n.s.)	+226.1% bimodal
Thermal Stress	+486.8%	+406.6%	+100.6%

Table 3: Summary of latency change by fault type and model family, confirming inverted profiles.

Vision models are thermal-sensitive but malformed-input-resilient. Both TFLite and ONNX runtimes process malformed inputs with bounded latency increases or no significant effect, because their tensor validation pipelines catch shape violations before expensive compute begins. Under thermal stress, however, parallel convolution workloads across all CPU cores cause immediate and severe clock reduction.

LLMs show the opposite. The GGUF runtime's autoregressive generation loop is less sensitive to thermal throttling (single-threaded, lower peak current draw), but has no guard against EOS misalignment under malformed input. That asymmetry has direct implications for production monitoring strategy.

5. Discussion

5.1 Why the Thermal API Fails

Budget OEM Android distributions implement `PowerManager.currentThermalStatus` conservatively. The callback was designed for system-level power management decisions, not application performance monitoring. On the Tecno CH7n running HiOS, the OS links its high-level thermal status to extreme battery temperature thresholds rather than real-time SoC junction register values. DVFS clock scaling happens well below that threshold, producing 5x latency regressions that the API does not surface. Application-layer latency measurement, not OS thermal callbacks, is the only reliable way to detect this on non-rooted consumer devices.

5.2 Why LLMs Fail on Malformed Input

Autoregressive generation loops are stateful in a way that vision inference pipelines are not. A single corrupted input to a CNN degrades that forward pass in proportion to the

corruption. A corrupted prompt to a language model can cause the tokenizer to produce token sequences where EOS never appears in the first N predicted tokens, causing the generation loop to run to the context-window boundary. The result is not proportional degradation but a binary outcome: either the model generates normally, or it runs for 20 seconds. Input sanitization and per-inference timeout enforcement are therefore both necessary for production LLM deployments.

5.3 Practical Implications

For vision pipeline developers: standard OS thermal APIs are insufficient. Pair any vision inference loop with application-layer latency tracking (EdgePulse or equivalent). Alert when rolling mean latency exceeds 2x baseline, not when the OS thermal status changes.

For LLM pipeline developers: thermal monitoring is less urgent than input validation. Enforce strict input sanitization before tokenization. Add a hard per-inference timeout (a value of 10 seconds covers the normal distribution; 20 seconds is the pathological cluster boundary observed here). Log any inference that exceeds 2x the baseline median for manual review.

6. Threats to Validity

1. **Single device.** All measurements were collected on one Tecno CH7n handset (Helio G35). Flagship SoCs with active thermal management or dedicated NPUs will likely show different thermal profiles. The blind-spot finding is specific to budget OEM Android distributions.
2. **Sample size for GGUF.** $N = 10$ per scenario for TinyLlama is too small to achieve statistical significance for the malformed-input result despite a strong effect. Larger N will produce tighter confidence intervals on the bimodal outlier frequency.
3. **Synthetic fault injection.** SATE AI generates malformed inputs programmatically and simulates memory pressure via heap allocation. Real-world sensor corruption and OS-level memory pressure may produce different distribution shapes.
4. **Thermal stress method.** Thermal stress was induced via back-to-back inference loops rather than a temperature-controlled chamber. Results reflect the specific thermal accumulation rate of the Helio G35 under this workload and may not generalise to different ambient temperatures or sustained production loads.

7. Conclusion

This paper reported the first empirical study combining software fault injection (SATE AI) and runtime observability (EdgePulse) on physical mobile hardware. Three findings

emerged from 280 real-device traces across TFLite, ONNX Runtime, and llama.cpp GGUF:

1. Android's `PowerManager.currentThermalStatus` API reported nominal in 100% of runs where vision model latency degraded by 4-5x. Application-layer timing is required to detect DVFS throttling on budget OEM devices.
2. TinyLlama GGUF under malformed input shows a bimodal failure mode: 80% of runs complete normally, 20% block for 20 seconds. Input sanitization and generation timeouts are both necessary for production LLM deployments.
3. Vision models and LLMs have opposite failure signatures. Monitoring and hardening strategies must be model-family-specific.

Both tools are open-source and available on pub.dev. Future work will expand the device testbed to include flagship SoCs, increase GGUF sample sizes to $N = 30$ per scenario, and test additional model families (Whisper, CLIP, Gemma).

References

- [1] Ullah, M. A. (2026a). SATE AI: A fault injection and reliability engineering framework for on-device AI models in mobile applications. *Zenodo*. doi.org/10.5281/zenodo.23250418
- [2] Ullah, M. A. (2026b). EdgePulse: A runtime observability framework for quantized large language models on consumer edge devices. *Zenodo*. doi.org/10.5281/zenodo.23248718
- [3] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265-283. usenix.org/osdi16
- [4] Lee, J., et al. (2019). On-device neural network execution for mobile AI applications. *IEEE Circuits and Systems Magazine*, 19(2), 24-39. doi.org/10.1109/MCAS.2019.2909033
- [5] Frantar, E., et al. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *International Conference on Learning Representations (ICLR)*. arxiv.org/abs/2210.17323
- [6] Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR 2018*, 2704-2713. doi.org/10.1109/CVPR.2018.00286
- [7] Howard, A., et al. (2019). Searching for MobileNetV3. *ICCV 2019*, 1314-1324. doi.org/10.1109/ICCV.2019.00140
- [8] Zhang, P., et al. (2024). TinyLlama: An open-source small language model. *arXiv preprint*. arxiv.org/abs/2401.02385
- [9] Wang, X., et al. (2024). Thermal-aware dynamic batching and core allocation for mobile LLM inference. *ACM Transactions on Embedded Computing Systems*, 23(4), 1-22. doi.org/10.1145/3649871
- [10] He, K., et al. (2016). Deep residual learning for image recognition. *CVPR 2016*, 770-778. doi.org/10.1109/CVPR.2016.90

- [11] Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *ICLR 2016*. arxiv.org/abs/1510.00149
- [12] Dutta, S., et al. (2021). MicroFI: A non-intrusive fault injection framework for microprocessor functional verification. *IEEE TCAD*, 40(6), 1102-1115. doi.org/10.1109/TCAD.2020.3013000
- [13] David, R., et al. (2021). TensorFlow Lite Micro: Embedded machine learning on TinyML systems. *MLSys 2021*, vol. 3, 800-811. arxiv.org/abs/2010.08678
- [14] Xu, D., et al. (2024). Characterizing reliability and resource contention in edge AI deployments. *IEEE Transactions on Mobile Computing*, 23(1), 115-130. doi.org/10.1109/TMC.2023.3289100